

PART 1

Model-View-Controller and
Object-Relation Mapping

Model-View-Controller (MVC)

MVC is a **software architecture pattern** that divides a software into **three interconnected parts**.

Internal storage of **data is separated from the way it is presented** and used by the user.

Model-View-Controller (MVC)

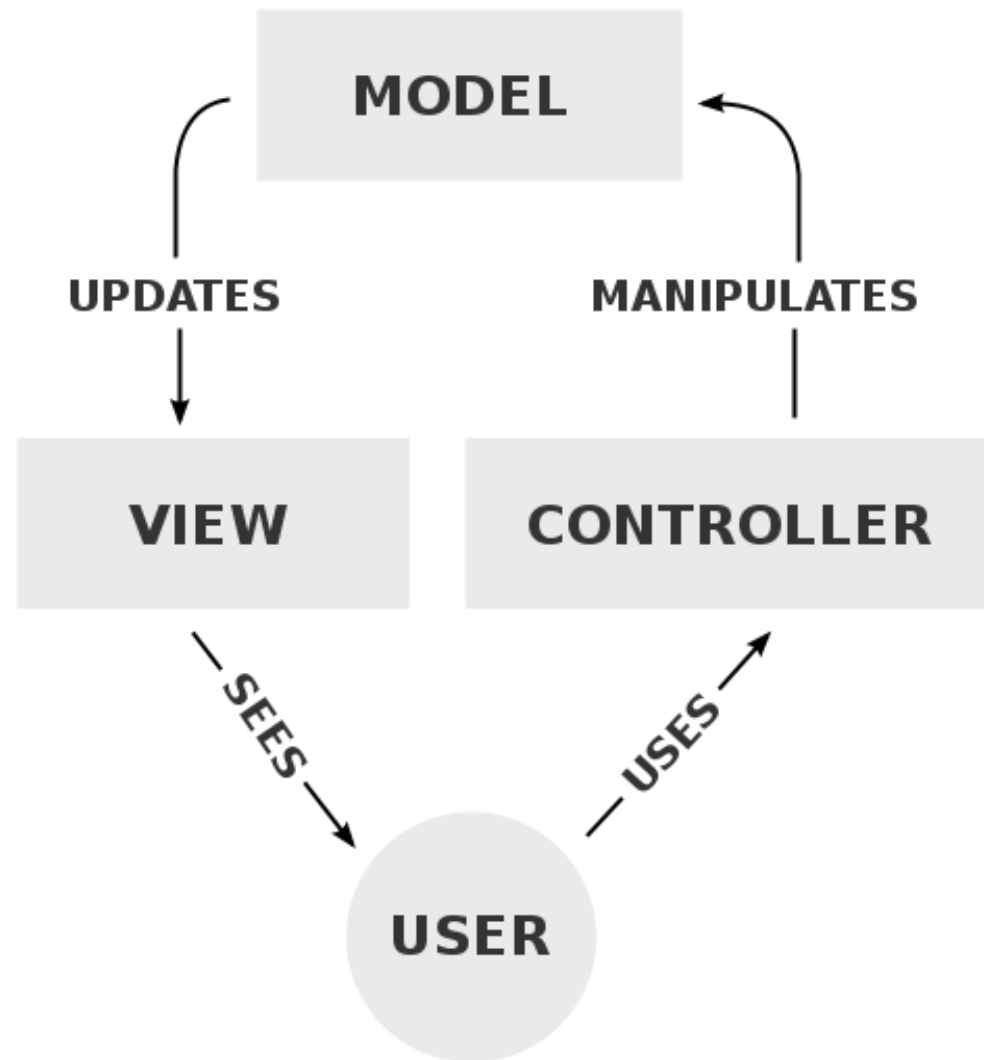
- Separation of
 - data and business *logic* (Model)
 - data *presentation* (View)
 - data *interaction* (Controller)
- **Model**: Responsible for managing the data, logic, and rules of the application.
- **View**: Responsible for displaying the user interface and presenting data to the user.
- **Controller**: Responsible for handling user input, updating the model, and selecting the view to display.

MVC

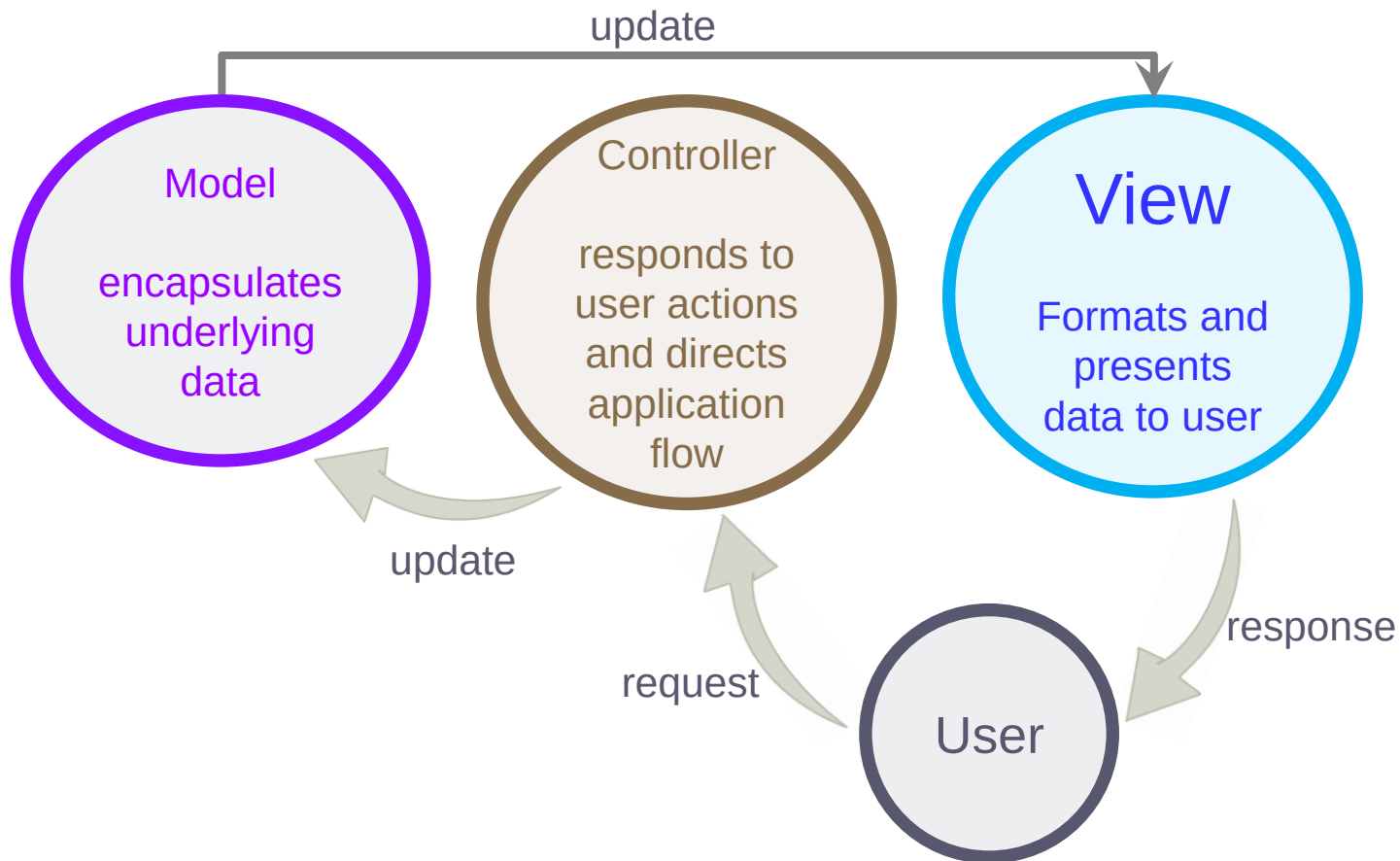
A **controller** can send commands to the model to update the model's state (e.g., editing a document). It can also send commands to its associated view to change the view's presentation of the model (e.g., by scrolling through a document).

A **model** notifies its associated views and controllers when there has been a change in its state. This notification allows the views to produce updated output, and the controllers to change the available set of commands. In some cases an MVC implementation might instead be "passive," so that other components must poll the model for updates rather than being notified.

A **view** requests information from the model that it needs for generating an output representation to the user.

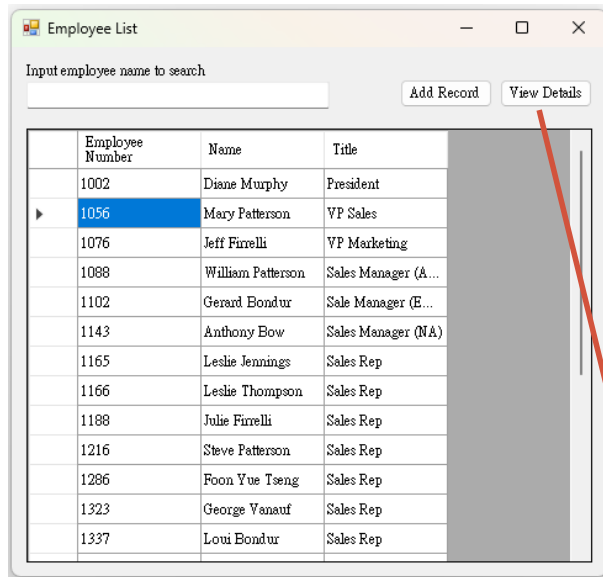


MVC Concepts and Steps



MVC in C# Applications

View

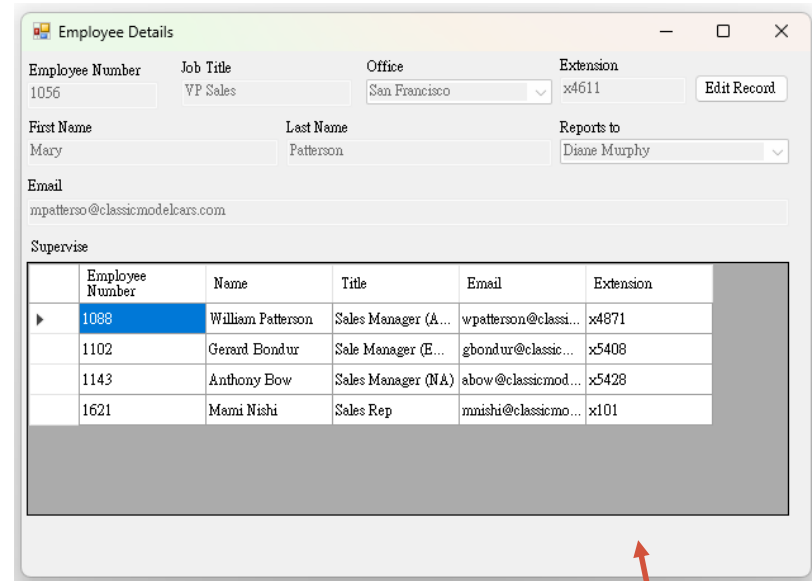


Employee List

Input employee name to search:

	Employee Number	Name	Title
	1002	Diane Murphy	President
▶	1056	Mary Patterson	VP Sales
	1076	Jeff Firrelli	VP Marketing
	1088	William Patterson	Sales Manager (A...
	1102	Gerard Bondur	Sale Manager (E...
	1143	Anthony Bow	Sales Manager (NA)
	1165	Leslie Jennings	Sales Rep
	1166	Leslie Thompson	Sales Rep
	1188	Julie Firrelli	Sales Rep
	1216	Steve Patterson	Sales Rep
	1286	Foon Yue Tseng	Sales Rep
	1323	George Vanauf	Sales Rep
	1337	Loui Bondur	Sales Rep

View



Employee Details

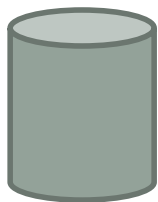
Employee Number: 1056 Job Title: VP Sales Office: San Francisco Extension: x4611

First Name: Mary Last Name: Patterson Reports to: Diane Murphy

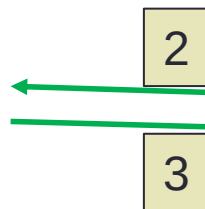
Email: mpatterso@classicmodelcars.com

Supervise

	Employee Number	Name	Title	Email	Extension
▶	1088	William Patterson	Sales Manager (A...	wpatterson@classi...	x4871
	1102	Gerard Bondur	Sale Manager (E...	gbondur@classic...	x5408
	1143	Anthony Bow	Sales Manager (NA)	abow@classicmod...	x5428
	1621	Mami Nishi	Sales Rep	mnishi@classicmo...	x101



Model



1

1 reference

```
private void btnView_Click(object sender, EventArgs e) { ... }
```

Controller

4

MVC in C# Applications

View

Employee Details

Employee Number: 1056 Job Title: VP Sales Office: San Francisco Extension: x4611

First Name: Mary Last Name: Patterson Reports to: Diane Murphy

Email: mpatterson@classicmodelcars.com

Supervise

	Employee Number	Name	Title	Email	Extension
▶	1088	William Patterson	Sales Manager (A...	wpatterson@classi...	x4871
	1102	Gerard Bondur	Sale Manager (E...	gbondur@classic...	x5408
	1143	Anthony Bow	Sales Manager (NA)	abow@classicmod...	x5428
	1621	Mami Nishi	Sales Rep	mnishi@classicmo...	x101

1

View

Employee Details

Employee Number: 1056 Job Title: VP Sales Office: San Francisco Extension: x4611

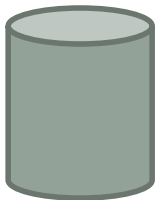
First Name: Mary Last Name: Patterson Reports to: Diane Murphy

Email: mpatterson@classicmodelcars.com

Supervise

	Employee Number	Name	Title	Email	Extension
▶	1088	William Patterson	Sales Manager (A...	wpatterson@classi...	x4871
	1102	Gerard Bondur	Sale Manager (E...	gbondur@classic...	x5408
	1143	Anthony Bow	Sales Manager (NA)	abow@classicmod...	x5428
	1621	Mami Nishi	Sales Rep	mnishi@classicmo...	x101

2



Model

```
1 reference
private void btnEdit_Click(object sender, EventArgs e)
{
    toggleInput(true);
}
```

Controller

Model = Database Table?

- In program, information can be stored by classes.
- In [relational] database, information are stored as relation.

```
public class Pet {
```

```
    private int id;  
    private String name;  
    private String specie;  
    private String owner;
```

```
    ...
```

```
}
```

Pet table

Id	Name	Specie	owner
1	Fluffy	Cat	Peter
2	Fatty	Cat	John

Model = Database Table?

- How to link up classes and database table?
- We need a **repository** class, which acts as an intermediary between the data source and the rest of the application.

```
public class Pet {  
  
    private int id;  
    private String name;  
    private String specie;  
    private String owner;  
  
    ...  
}
```

a class representing a
record in the Pet table

```
public class PetRepo {  
  
    public List<Pet> getPets() { }  
    public Pet getByld(int id){ }  
  
    public boolean addRecord(Pet p) { }  
    public boolean deleteRecord(int id) { }  
    public boolean updateRecord(Pet p) { }  
}
```

Exposes methods to
interact with database

Object-Relational Mapping (ORM)

- ORM is a programming technique used to convert data between incompatible systems in object-oriented programming languages and relational databases.
- Popular ORM frameworks:
 - **Entity Framework** (C#/.NET)
 - **Hibernate** (Java)
 - **Django ORM** (Python)
 - **Sequelize** (Node.js)
 - **SQLAlchemy** (Python)

Key aspects of ORM

- **Mapping**

- ORM frameworks map database tables to classes, table rows to objects, and columns to object attributes, making it easier for developers to work with data in an object-oriented way.

- **Abstraction**

- ORM abstracts away the complexities of SQL queries, allowing developers to interact with the database using programming language constructs.

- **CRUD Operations**

- ORM frameworks typically provide methods for performing **CRUD** operations (**C**reate, **R**ead, **U**ppdate, **D**eleate) on database records using object-oriented syntax.

- **Relationships**

- ORM frameworks facilitate defining and managing relationships between database tables/entities in an object-oriented manner.

Key characteristics of a repository class

- **Data Access Abstraction**

- The repository class encapsulates the logic required to access data from the underlying data source. This allows the rest of the application to interact with the repository without needing to know the specifics of how data is retrieved or stored.

- **CRUD Operations**

- Typically, a repository class provides methods to perform CRUD (Create, Read, Update, Delete) operations on data entities. These methods abstract away the details of data manipulation and provide a clean interface for interacting with data.

- **Single Responsibility Principle**

- The repository pattern helps in adhering to the Single Responsibility Principle by separating the data access logic from the business logic of the application. This makes the codebase more maintainable and easier to test.

- **Encapsulation of Query Logic**

- Repository classes can encapsulate complex query logic, allowing developers to centralize and manage data access logic in a structured manner. This can help in maintaining a consistent approach to data access throughout the application.

- **Mocking for Testing**

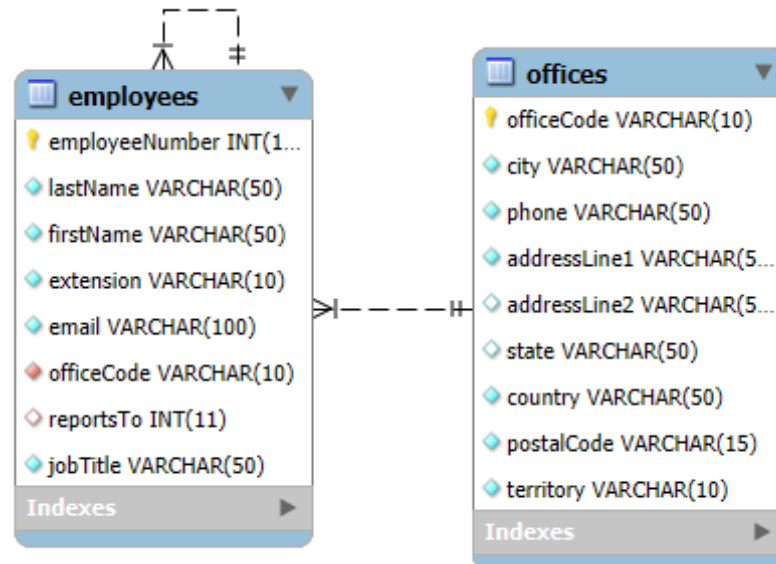
- Repository classes can be easily mocked or replaced with fake implementations during unit testing, enabling developers to test components that depend on data access logic without actually interacting with a database or external services.

PART 2

Implementation of a simple MVC application

MySQL Database and Tables

- We will use the classicmodels (provided by mysqltutorial)
 - <https://www.mysqltutorial.org/getting-started-with-mysql/mysql-sample-database/>
- We will just use two tables:
 - employees
 - offices
- The ER diagram



Step 1: Create Entity Class

```
public class Employee
{
    private int employeeNumber;
    private string lastName;
    private string firstName;
    private string extension;
    private string email;
    private string jobTitle;

    private string officeCode;

    private int reportsTo;

    //Constructor
    public Employee() { }

    //Getter and Setter Methods
    public int OfficeCode
    {
        get { return officeCode; } set { officeCode = value; }
    }
}
```

Foreign keys and
the object associated with the foreign key

Step 1: Create Entity Class

//Other getter and setter methods are skipped

```
public string FullName
{
    get { return firstName + " " + lastName; }
}
```

```
}
```

Derived attribute

Not stored directly in database or class.

But can be computed from other attributes/properties.

Common example:

- Full name
- Age

Task 1: Complete the Entity class for the table Offices

Step 2: Create Repository Class

```
public class EmployeeRepo
{
    //Create
    public bool Add(Employee employee) { }

    //Read – Normally we need more than one R methods
    public List<Employee> GetAll() { }
    public Employee GetById(int id) { }
    public List<Employee> GetByName(string name) { }
    public List<Employee> GetSupervise(int id) { }

    //Update
    public bool Edit(Employee employee) { }

    //Delete
    public bool Delete(int id) { }
}
```

Typical Database Programming

1. Establishes a connection to the database
2. Creates and executes SQL statements
3. Processes the results
4. Close database connection

To avoid redundant code, we write a standalone function to get the MySqlConnection object in a separate class.

```
static class Program
{
    public static string host = "127.0.0.1";
    public static string db = "classicmodels";
    public static string userId = "root";
    public static string password = "";

    public static MySqlConnection GetConnection()
    {
        string connStr = $"Server={host};Database={db};User ID={userId};Password={password};Pooling=true;";

        return new MySqlConnection(connStr);
    }
    ...
}
```

Step 2: Create Repository Class

```
public List<Employee> GetAll() {  
1  MySqlConnection conn = Program.GetConnection();  
   conn.Open();  
  
2  MySqlCommand sql = new MySqlCommand("SELECT * FROM employees", conn);  
   MySqlDataReader reader = sql.ExecuteReader();  
   Use ExecuteReader() for R operations  
  
3  List<Employee> employees = new List<Employee>();  
   while (reader.Read()) {  
       Employee employee = new Employee();  
       employee.EmployeeNumber = reader.GetInt32("employeeNumber");  
       employee.LastName = reader.GetString("lastName");  
       employee.FirstName = reader.GetString("firstName");  
       employee.Extension = reader.GetString("extension");  
       employee.Email = reader.GetString("email");  
       employee.OfficeCode = reader.GetString("officeCode");  
       employee.JobTitle = reader.GetString("jobTitle");  
       employee.ReportsTo = (reader.IsDBNull(reader.GetOrdinal("reportsTo")) ?  
                           -1 : reader.GetInt32("reportsTo"));  
       employees.Add(employee);  
   }  
4  conn.Close();  
   return employees;  
}
```

Step 2: Create Repository Class

```
public Employee GetById(int id) {  
1  MySqlConnection conn = Program.GetConnection();  
   conn.Open();  
  
2  MySqlCommand sql = new MySqlCommand  
    ( "SELECT * FROM employees WHERE employeeNumber = @id", conn);  
  
   sql.Parameters.AddWithValue("@id", id);  
  
3  MySqlDataReader reader = sql.ExecuteReader();  
   Employee employee = null;  
   if (reader.Read())  
   { ... }  
  
4  conn.Close();  
   return employee;  
}
```

@id is a placeholder

We set some value to replace the placeholder.
By this approach, we can avoid special characters by users,
which may breaks our SQL statement.

Step 2: Create Repository Class

```
public bool Add(Employee employee)
{
    MySqlConnection conn = Program.GetConnection();
    conn.Open();

    MySqlCommand sql = new MySqlCommand("INSERT INTO employees (employeeNumber,
lastName, firstName, extension, email, officeCode, jobTitle, reportsTo) VALUES
(@employeeNumber, @lastName, @firstName, @extension, @email, @officeCode,
@jobTitle, @reportsTo)", conn);
    sql.Parameters.AddWithValue("@employeeNumber", employee.EmployeeNumber);
    sql.Parameters.AddWithValue("@lastName", employee.LastName);
    sql.Parameters.AddWithValue("@firstName", employee.FirstName);
    sql.Parameters.AddWithValue("@extension", employee.Extension);
    sql.Parameters.AddWithValue("@email", employee.Email);
    sql.Parameters.AddWithValue("@officeCode", employee.OfficeCode);
    sql.Parameters.AddWithValue("@jobTitle", employee.JobTitle);
    sql.Parameters.AddWithValue("@reportsTo",
        employee.ReportsToId < 0 ? (object)DBNull.Value : employee.ReportsToId);

    int result = sql.ExecuteNonQuery();
    conn.Close();

    return result == 1;
}
```

Use ExecuteNonQuery() for non-R operations

Step 2: Create Repository Class

• Task 2A

- Complete the remaining methods for EmployeeRepo
- `public List<Employee> GetByName(string name)`
 - `SELECT * FROM employees WHERE lastName LIKE @name OR firstName LIKE @name`
- `public List<Employee> GetSupervise(int id)`
 - `SELECT * FROM employees WHERE reportsTo = @id`
- `public bool Edit(Employee employee)`
 - `UPDATE employees SET lastName = @lastName, firstName = @firstName, extension = @extension, email = @email, officeCode = @officeCode, jobTitle = @jobTitle, reportsTo = @reportsToId WHERE employeeNumber = @employeeNumber`
- `public bool Delete(int id)`
 - `DELETE FROM employees WHERE employeeNumber = @id`

Step 2: Create Repository Class

- **Task 2B**

- Write a repository class for Office

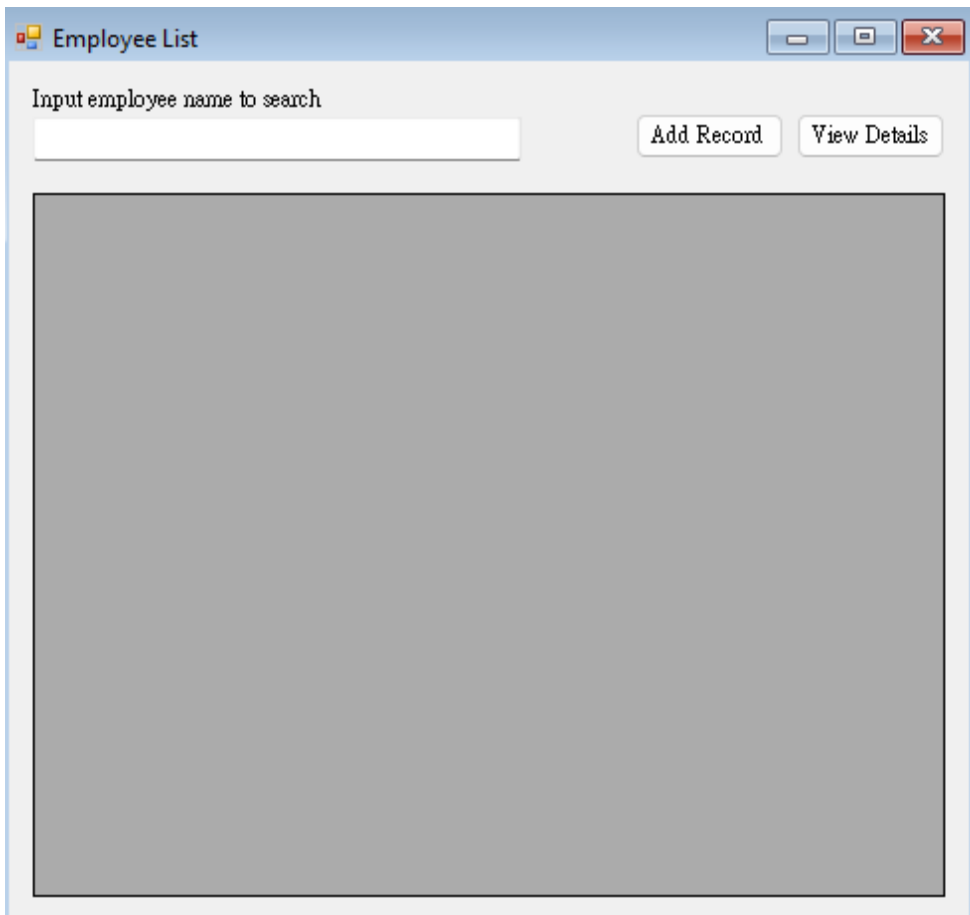
```
public class OfficeRepo
{
    //Create
    public bool Add(Office office) { }

    //Read
    public List<Office> GetAll() { }
    public Office GetById(int id) { }

    //Update
    public bool Edit(Office office) { }

    //Delete
    public bool Delete(string id) { }
}
```

Step 3a: Create View (EmployeeView)



The screenshot shows a Windows application window titled "Employee List". Inside the window, there is a label "Input employee name to search" followed by a text box. To the right of the text box are two buttons: "Add Record" and "View Details". Below these elements is a large, empty gray rectangular area, which is the DataGridView.

1 Label
1 TextBox
2 Buttons
1 DataGridView

The DataGridView will show the information of employees

Step 3a: Create View (EmployeeView)

```
private EmployeeRepo employeeRepo = new EmployeeRepo();
public EmployeeView() {
    InitializeComponent();
    refreshDataGrid();
}
```

```
public void refreshDataGrid() {
```

1. Get employee list form repository

```
    List<Employee> employees = employeeRepo.GetByName(txtName.Text);
```

```
    dgvEmployee.Columns.Clear();
    dgvEmployee.DataSource = null;
```

2. Clear old data in data grid view

```
    dgvEmployee.AutoGenerateColumns = false;
    dgvEmployee.Columns.Add(new DataGridViewTextBoxColumn {
        HeaderText = "Employee Number",
        DataPropertyName = "employeeNumber",
        Name = "employeeNumber" });
```

```
    dgvEmployee.Columns.Add(new DataGridViewTextBoxColumn {
        HeaderText = "Name",
        DataPropertyName = "fullName",
        Name = "fullName" });
```

3. Set what to be displayed in data grid view

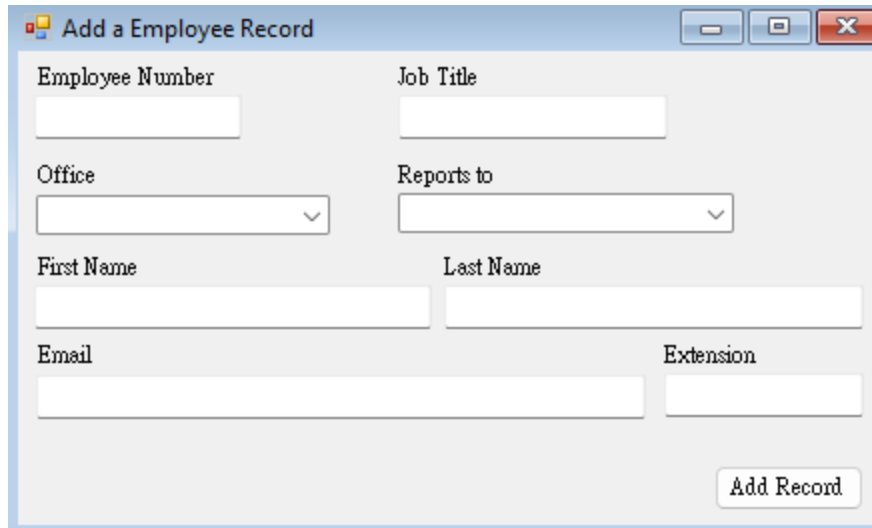
```
    dgvEmployee.Columns.Add(new DataGridViewTextBoxColumn {
        HeaderText = "Title",
        DataPropertyName = "jobTitle",
        Name = "jobTitle" });
```

```
    dgvEmployee.DataSource = employees;
```

4. Set the data source as the employee list

```
}
```

Step 3b: Create View (AddEmployeeForm)



The screenshot shows a Windows-style window titled "Add a Employee Record". The window contains a form with the following fields and controls:

- Employee Number:** A text box.
- Job Title:** A text box.
- Office:** A dropdown menu.
- Reports to:** A dropdown menu.
- First Name:** A text box.
- Last Name:** A text box.
- Email:** A text box.
- Extension:** A text box.
- Add Record:** A button located at the bottom right of the form.

8 Labels
6 TextBoxes
2 ComboBoxes
1 Button

Step 3b: Create View (AddEmployeeForm)

```
private EmployeeRepo employeeRepo = new EmployeeRepo();
private OfficeRepo officeRepo = new OfficeRepo();
private EmployeeView parent;

public AddEmployeeForm(EmployeeView parent)
{
    InitializeComponent();
    this.parent = parent;

    List<Employee> employees = employeeRepo.GetAll();
    employees = employees.OrderBy(e => e.FullName).ToList();
    employees.Insert(0, new Employee { EmployeeNumber = -1, LastName = "NONE" });
    cboReportsTo.DataSource = employees;
    cboReportsTo.DisplayMember = "FullName";
    cboReportsTo.ValueMember = "employeeNumber";

    List<Office> offices = officeRepo.GetAll();
    offices = offices.OrderBy(o => o.City).ToList();
    cboOffice.DataSource = offices;
    cboOffice.DisplayMember = "city";
    cboOffice.ValueMember = "officeCode";
}
```

As there may be no one the employee needs to report to, we add a NONE choice at the top of the combo box.

Setting up the combo boxes

1. Get list from repository
2. Sort list by alphabetic order
3. Set data source
4. Set which property to be displayed
5. Set the value received in program

Step 3c: Create View (EmployeeDetailsView)

The screenshot shows a Windows application window titled "Employee Details". The form contains the following fields and controls:

- Employee Number: Text box
- Job Title: Text box
- Office: Combo box
- Extension: Text box
- First Name: Text box
- Last Name: Text box
- Reports to: Combo box
- Email: Text box
- Supervise: A large, empty rectangular area, likely a DataGridView.
- Buttons: "Edit Record", "Delete Record", "Cancel Edit", and "Save Record".

Red arrows point from the "Visible = false" box to the "Delete Record", "Cancel Edit", and "Save Record" buttons.

9 Labels
6 TextBoxes
2 ComboBoxes
4 Buttons
1 DataGridView

All TextBoxes and ComboBoxes
Enable = false

Visible = false

Step 3c: Create View (EmployeeDetailsView)

```
private readonly int employeeNum;
private EmployeeRepo employeeRepo = new EmployeeRepo();
private OfficeRepo officeRepo = new OfficeRepo();

public EmployeeDetailsView(int employeeNum)
{
    InitializeComponent();
    this.employeeNum = employeeNum;
    refreshData();
}

private void refreshData()
{
    List<Employee> employees = employeeRepo.GetAll();

    employees = employees.OrderBy(e => e.FullName).ToList();
    employees.Remove(employees.FirstOrDefault(e => e.EmployeeNumber == employeeNum));
    employees.Insert(0, new Employee { EmployeeNumber = -1, LastName = "NONE" });
    cboReportsTo.DataSource = employees;
    cboReportsTo.DisplayMember = "FullName";
    cboReportsTo.ValueMember = "employeeNumber";
    List<Office> offices = officeRepo.GetAll();
    offices = offices.OrderBy(o => o.City).ToList();
    cboOffice.DataSource = offices;
    cboOffice.DisplayMember = "city";
    cboOffice.ValueMember = "officeCode";
    Employee employee = employeeRepo.GetById(employeeNum);
```

No one should supervise himself, so we remove his record from the staff list, before setting it as the data source to the combo box.

Step 3c: Create View (EmployeeDetailsView)

```

if (employee != null) {
    txtEmployeeNum.Text = employee.EmployeeNumber.ToString();
    txtLastName.Text = employee.LastName;
    txtFirstName.Text = employee.FirstName;
    txtTitle.Text = employee.JobTitle;
    txtExt.Text = employee.Extension;
    txtEmail.Text = employee.Email;
    cboOffice.SelectedValue = employee.OfficeCode;
    cboReportsTo.SelectedValue = employee.ReportsToId;

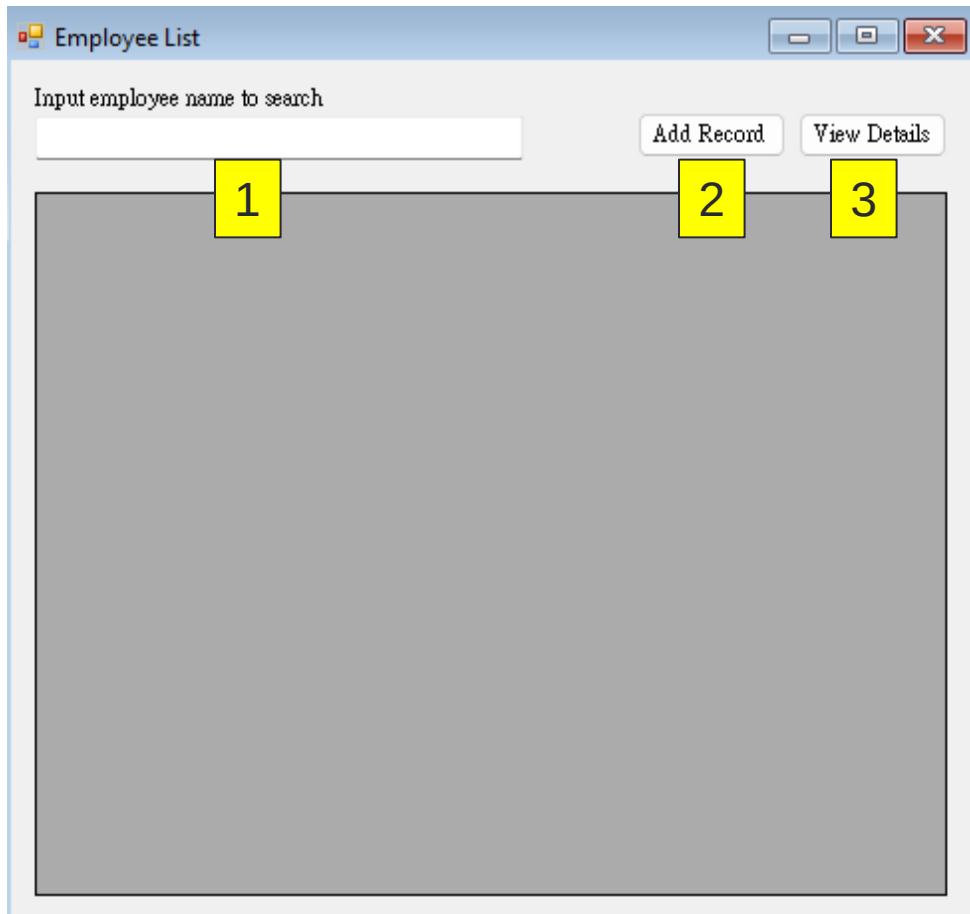
    List<Employee> supervise = employeeRepo.GetSupervise(employee.EmployeeNumber);
    dvgSupervise.Columns.Clear();
    dvgSupervise.AutoGenerateColumns = false;

    dvgSupervise.Columns.Add(new DataGridViewTextBoxColumn {
        HeaderText = "Employee Number", DataPropertyName = "employeeNumber",
        Name = "employeeNumber" });
    dvgSupervise.Columns.Add(new DataGridViewTextBoxColumn {
        HeaderText = "Name", DataPropertyName = "fullName", Name = "fullName" });
    dvgSupervise.Columns.Add(new DataGridViewTextBoxColumn {
        HeaderText = "Title", DataPropertyName = "jobTitle", Name = "jobTitle" });
    dvgSupervise.Columns.Add(new DataGridViewTextBoxColumn {
        HeaderText = "Email", DataPropertyName = "email", Name = "email" });
    dvgSupervise.Columns.Add(new DataGridViewTextBoxColumn {
        HeaderText = "Extension", DataPropertyName = "extension", Name = "extension" });

    dvgSupervise.DataSource = supervise;
}
}

```

Step 4a: Add Event Handler



Step 4a: Add Event Handler

1. When user type something, the employee details in data grid view will change accordingly.

```
private void txtName_TextChanged(object sender, EventArgs e) {  
    refreshDataGrid();  
}
```

2. When user click the button, a new form will be displayed for user to input a new employee record.

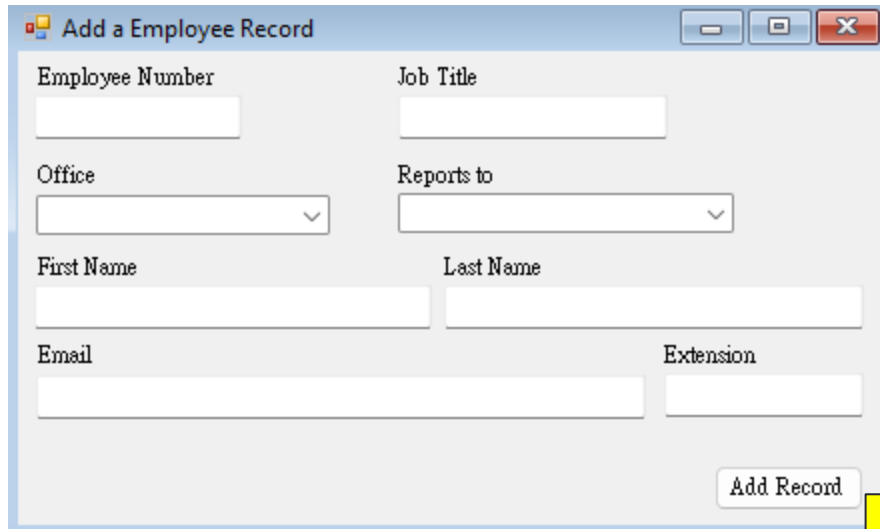
```
private void btnAdd_Click(object sender, EventArgs e) {  
    AddEmployeeForm form = new AddEmployeeForm(this);  
    form.Show();  
}
```


Step 4a: Add Event Handler

3. When user click the button, the selected employee information will be displayed in a new Window.

```
private void btnView_Click(object sender, EventArgs e) {  
    if (dgvEmployee.SelectedCells.Count > 0) {  
        int selectedRowIndex = dgvEmployee.SelectedCells[0].RowIndex;  
        DataGridViewRow selectedRow = dgvEmployee.Rows[selectedRowIndex];  
        int employeeNum = Convert.ToInt32(selectedRow.Cells["employeeNumber"].Value.ToString());  
  
        if (employeeRepo.GetById(employeeNum) != null) {  
            EmployeeDetailsView view = new EmployeeDetailsView(employeeNum);  
            view.Show();  
        } else {  
            MessageBox.Show("Employee not found", "Error",  
                            MessageBoxButtons.OK, MessageBoxIcon.Error);  
            refreshDataGrid();  
        }  
    } else {  
        MessageBox.Show("Please select a employee to view.", "No Selection",  
                        MessageBoxButtons.OK, MessageBoxIcon.Warning);  
    }  
}
```

Step 4b: Add Event Handler



A screenshot of a Windows-style dialog box titled "Add a Employee Record". The dialog box contains several input fields arranged in a grid-like fashion. At the top right are standard window controls (minimize, maximize, close). The fields are: "Employee Number" (text box), "Job Title" (text box), "Office" (dropdown menu), "Reports to" (dropdown menu), "First Name" (text box), "Last Name" (text box), "Email" (text box), and "Extension" (text box). At the bottom right is a button labeled "Add Record".

1

Step 4b: Add Event Handler

1. When user click the button, a new employee record will be added to database.

```
private void btnAdd_Click(object sender, EventArgs e) {
    Employee employee = employeeRepo.GetById(Convert.ToInt32(txtEmployeeNum.Text));
    if(Convert.ToInt32(txtEmployeeNum.Text) <= 0) {
        MessageBox.Show("Employee number must be greater than 0", "Error",
            MessageBoxButtons.OK, MessageBoxIcon.Error);

        return;
    } else if (employee != null) {
        MessageBox.Show("Employee ID already used", "Error",
            MessageBoxButtons.OK, MessageBoxIcon.Error);
    } else {
        employee = new Employee();
        employee.EmployeeNumber = Convert.ToInt32(txtEmployeeNum.Text);
        employee.LastName = txtLastName.Text;
        employee.FirstName = txtFirstName.Text;
        employee.JobTitle = txtTitle.Text;
        employee.Extension = txtExt.Text;
        employee.Email = txtEmail.Text;
        employee.OfficeCode = Convert.ToString(cboOffice.SelectedValue);
        employee.ReportsToId = Convert.ToInt32(cboReportsTo.SelectedValue);
        employeeRepo.Add(employee);
        Close();
        parent.refreshDataGrid();
    }
}
```

Step 4c: Add Event Handler

The image shows a software window titled "Employee Details" with standard Windows window controls (minimize, maximize, close). The form contains several input fields and buttons. A yellow box with the number "1" points to the "Edit Record" button. A yellow box with the number "2" points to the "Delete Record" button. A yellow box with the number "3" points to the "Cancel Edit" button. A yellow box with the number "4" points to the "Save Record" button.

Employee Details

Employee Number Job Title Office Extension **1** Edit Record

First Name Last Name Reports to

Email

Supervise

Delete Record **2** Cancel Edit **3** Save Record **4**

Step 4c: Add Event Handler

1. Originally, user can only see the employee details. When user press the edit button, user can now change the details. The delete record, cancel edit, and save record buttons will become visible. The edit button will become invisible.

```
private void btnEdit_Click(object sender, EventArgs e) {  
    toggleInput(true);  
}  
  
private void toggleInput(bool enable) {  
    txtLastName.Enabled = enable;    txtFirstName.Enabled = enable;  
    txtTitle.Enabled = enable;    txtExt.Enabled = enable;  
    txtEmail.Enabled = enable;    cboOffice.Enabled = enable;  
    cboReportsTo.Enabled = enable;    btnSave.Visible = enable;  
    btnCancel.Visible = enable;    btnDel.Visible = enable;  
    btnEdit.Visible = !enable;  
}
```

Step 4c: Add Event Handler

2. When the delete record button is pressed, the employee record will be deleted.

```
private void btnDel_Click(object sender, EventArgs e) {
    List<Employee> supervise = employeeRepo.GetSupervise(employeeNum);
    if(supervise.Count > 0) {
        MessageBox.Show("Cannot delete employee with supervisees", "Error",
            MessageBoxButtons.OK, MessageBoxIcon.Error);

        return;
    }
    DialogResult result = MessageBox.Show("Are you sure you want to delete this employee?",
        "Confirm Delete", MessageBoxButtons.YesNo, MessageBoxIcon.Warning);

    if (result == DialogResult.Yes) {
        if (employeeRepo.Delete(employeeNum)) {
            MessageBox.Show("Employee deleted successfully", "Success",
                MessageBoxButtons.OK, MessageBoxIcon.Information);

            this.Close();
        } else {
            MessageBox.Show("Error deleting employee", "Error",
                MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
    }
}
```

Step 4c: Add Event Handler

3. When the cancel edit button is pressed, user will not be allowed to make changes to the employee details. All changes will be discarded and the form will return to readonly status.

```
private void btnCancel_Click(object sender, EventArgs e)
{
    DialogResult result = MessageBox.Show("Are you sure you want to cancel?",
        "Confirm Cancel", MessageBoxButtons.YesNo, MessageBoxIcon.Warning);
    if (result == DialogResult.Yes)
    {
        toggleInput(false);
        refreshData();
        MessageBox.Show("Changes cancelled", "Cancelled",
            MessageBoxButtons.OK, MessageBoxIcon.Information);
    }
}
```

Step 4c: Add Event Handler

- When the save record button is pressed, changes will be saved to database. The form will return to readonly status.

```
private void btnSave_Click(object sender, EventArgs e) {
    DialogResult result = MessageBox.Show("Are you sure you want to save?", "Confirm",
        MessageBoxButtons.YesNo, MessageBoxIcon.Warning);
    if (result == DialogResult.Yes) {
        Employee employee = new Employee();
        employee.EmployeeNumber = Convert.ToInt32(txtEmployeeNum.Text);
        employee.LastName = txtLastName.Text; employee.FirstName = txtFirstName.Text;
        employee.JobTitle = txtTitle.Text; employee.Extension = txtExt.Text;
        employee.Email = txtEmail.Text;
        employee.OfficeCode = Convert.ToString(cboOffice.SelectedValue);
        employee.ReportsToId = Convert.ToInt32(cboReportsTo.SelectedValue);
        if (employeeRepo.Edit(employee)) {
            MessageBox.Show("Employee updated successfully", "Success",
                MessageBoxButtons.OK, MessageBoxIcon.Information);
            toggleInput(false);
            refreshData();
        } else {
            MessageBox.Show("Error updating employee", "Error",
                MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
    }
}
```